**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н. Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

# Лабораторная работа №2
# по дисциплине «Анализ Алгоритмов»

**Тема** Алгоритм Винограда

**Студент** Платонова М. И.

**Группа** ИУ7-51Б

**Преподаватель** Волкова Л. Л.

Москва — 2024 г.

# СОДЕРЖАНИЕ

**ВВЕДЕНИЕ** . . . . . **4**

**1 Аналитическая часть** . . . . . **5**

1.1 Алгоритмы умножения матриц . . . . . 5

1.1.1 Стандартный алгоритм . . . . . 5

1.1.2 Алгоритм Винограда . . . . . 5

1.1.3 Оптимизированный алгоритм Винограда . . . . . 6

**2 Конструкторская часть** . . . . . **7**

2.1 Описание структуры ПО . . . . . 7

2.2 Описание алгоритмов . . . . . 7

2.3 Алгоритм взаимодействия с пользователем . . . . . 7

2.4 Модель вычисления трудоёмкости . . . . . 13

2.5 Трудоёмкость алгоритма умножения матриц . . . . . 13

2.5.1 Инициализация результирующей матрицы . . . . . 13

2.5.2 Трудоёмкость стандартного алгоритма умножения матриц . . . . . 13

2.5.3 Общая трудоёмкость . . . . . 14

2.6 Трудоёмкость алгоритма Винограда умножения матриц . . . . . 14

2.7 Трудоёмкость оптимизированного алгоритма Винограда умножения матриц . . . 15

2.7.1 Оптимизации в алгоритме . . . . . 15

2.8 Трудоёмкость оптимизированного алгоритма Винограда умножения матриц . . . 15

2.8.1 Оптимизации в алгоритме . . . . . 15

2.8.2 Трудоёмкость оптимизированного алгоритма Винограда . . . . . 16

**3 Технологическая часть** . . . . . **18**

3.1 Требования к программному обеспечению . . . . . 18

3.2 Средства реализации . . . . . 18

3.3 Структура программы . . . . . 18

3.4 Реализация алгоритмов . . . . . 18

3.5 Функциональные тесты . . . . . 23

**4 Исследовательская часть . . . . . 24**

4.1 Технические характеристики . . . . . 24

4.2 Сравнительный анализ времени выполнения алгоритмов . . . . . 24

**ЗАКЛЮЧЕНИЕ . . . . . 27**

**СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . . 28**

# ВВЕДЕНИЕ

Матрицы находят широкое применение в различных областях, начиная от программирования и заканчивая физикой и экономикой. В математике матрица представляет собой таблицу чисел, организованную в строки и столбцы. Операции с матрицами, такие как сложение, умножение и возведение в степень, играют важную роль в решении различных задач.

Алгоритм Копперсмита — Винограда является одной из известных методик умножения квадратных матриц. Этот алгоритм был предложен Д. Копперсмитом и Ш. Виноградом, и он имеет наилучшую асимптотику среди известных алгоритмов умножения матриц. Однако на практике этот метод не всегда эффективен из-за высокой константы пропорциональности, что делает его предпочтительным только для матриц размером от $1000 \times 1000$, превосходящих возможности оперативной памяти большинства современных компьютеров.

Целью данной лабораторной работы является исследование алгоритмов умножения матриц (стандартный, классический и оптимизированный алгоритмы Винограда).

Для достижения поставленной цели необходимо решить следующие задачи:

– рассмотреть и реализовать классический алгоритм умножения матриц, алгоритм Винограда и его оптимизированную версию;

– провести анализ временных затрат алгоритмов умножения матриц;

– выполнить анализ трудоёмкости алгоритмов;

– подготовить графики, отображающие результаты сравнительного анализа;

– разработать набор тестов для оценки работоспособности реализованных алгоритмов;

– сформулировать и представить результаты в отчёте о проведённой лабораторной работе.

# 1 Аналитическая часть

В этом разделе представлены алгоритмы стандартного умножения матриц, алгоритм Винограда и его оптимизированная версия [1].

## 1.1 Алгоритмы умножения матриц

### 1.1.1 Стандартный алгоритм

Имеются две матрицы, $A$ и $B$, которые необходимо перемножить. Пусть $A$ имеет размерность $M \times N$, а матрица $B$ — размерность $N \times K$. Результирующая матрица $C$ будет иметь размерность $M \times K$. Каждый элемент матрицы $C$ вычисляется по формуле (1.1):

$$C_{ij} = \sum_{r=1}^{N} A_{ir} B_{rj}, \quad i \in \{1, \ldots, M\}, \quad j \in \{1, \ldots, K\} \tag{1.1}$$

Стандартный алгоритм умножения матриц последовательно перемножает строки первой матрицы на столбцы второй, что позволяет получить элементы результирующей матрицы.

### 1.1.2 Алгоритм Винограда

Алгоритм Винограда – это метод умножения квадратных матриц, предложенный в 1987 году Д. Копперсмитом и Ш. Виноградом. Основная идея – уменьшить количество операций умножения для ускорения вычислений.

Алгоритм Винограда использует предварительную обработку строк и столбцов матриц для ускорения вычислений. Вместо прямого вычисления скалярного произведения, алгоритм разбивает его на несколько простых частей, что позволяет сократить количество умножений. Имеются два вектора $V = (v_1, v_2, v_3, v_4)$ и $W = (w_1, w_2, w_3, w_4)$. Тогда их скалярное произведение рассчитывается по формуле (1.2):

$$V \cdot W = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4. \tag{1.2}$$

Алгоритм Винограда преобразует это выражение следующим образом (1.3):

$$V \cdot W = (v_1 + w_2)(v_2 + w_1) + (v_3 + w_4)(v_4 + w_3) - v_1 v_2 - v_3 v_4 - w_1 w_2 - w_3 w_4. \tag{1.3}$$

Данное выражение требует большего числа операций по сравнению с классическим методом, но оно позволяет проводить предварительные вычисления для строк и столбцов матриц, уменьшая количество операций умножения для каждого элемента произведения матриц.

### 1.1.3 Оптимизированный алгоритм Винограда

Оптимизированный алгоритм Винограда [2] предназначен для уменьшения количества операций умножения, что делает его более эффективным для работы с матрицами с матрицами размером от $500 \times 500$ и более. Идея оптимизации заключается в том, что предварительно рассчитанные значения для строк первой матрицы и столбцов второй сохраняются, что позволяет их повторно использовать. Это сокращает общие затраты на умножения и сложения.

В оптимизированном алгоритме для матрицы размером $n \times n$ выполняется $n^2$ сложений и $n^2/2$ умножений, что делает его быстрее по сравнению с классическим алгоритмом. Если размер матрицы нечётный, требуется дополнительное умножение для завершения расчётов.

Эта оптимизация существенно влияет на производительность при работе с большими матрицами, где снижение количества операций умножения позволяет сэкономить время и ресурсы.

## Вывод

В данном разделе рассмотрены стандартный алгоритм умножения матриц, алгоритм Винограда и его оптимизированная версия. Алгоритм Винограда отличается от классического тем, что предварительная обработка строк и столбцов позволяет уменьшить количество операций умножения. Оптимизированная версия алгоритма также сокращает количество умножений и увеличивает скорость работы при умножении матриц размером от $500 \times 500$ и более.

# 2 Конструкторская часть

В данном разделе представлены описания алгоритмов для умножения матриц, а также описание типов данных и оценка трудоёмкости реализованных алгоритмов.

## 2.1 Описание структуры ПО

На вход алгоритмов поступают две матрицы $matr1$ и $matr2$, размеры которых вводятся пользователем. Пользователь может выбрать один из двух вариантов ввода данных: вручную или сгенерировать случайные матрицы. На выходе программа возвращает результат произведения матриц, записанный в $res_matr$.

## 2.2 Описание алгоритмов

Программа предоставляет выбор между тремя алгоритмами умножения матриц:

- стандартный алгоритм умножения матриц;
- умножение матриц по алгоритму Винограда;
- оптимизированный алгоритм Винограда.

## 2.3 Алгоритм взаимодействия с пользователем

Программа взаимодействует с пользователем через консольное меню:

- пользователь вводит размеры матриц;
- программа предлагает выбрать способ ввода: вручную или случайная генерация матриц;
- после ввода матриц пользователь может выбрать один из предложенных методов умножения.

Рисунки 2.1 — 2.5 представляют описание реализованных алгоритмов.

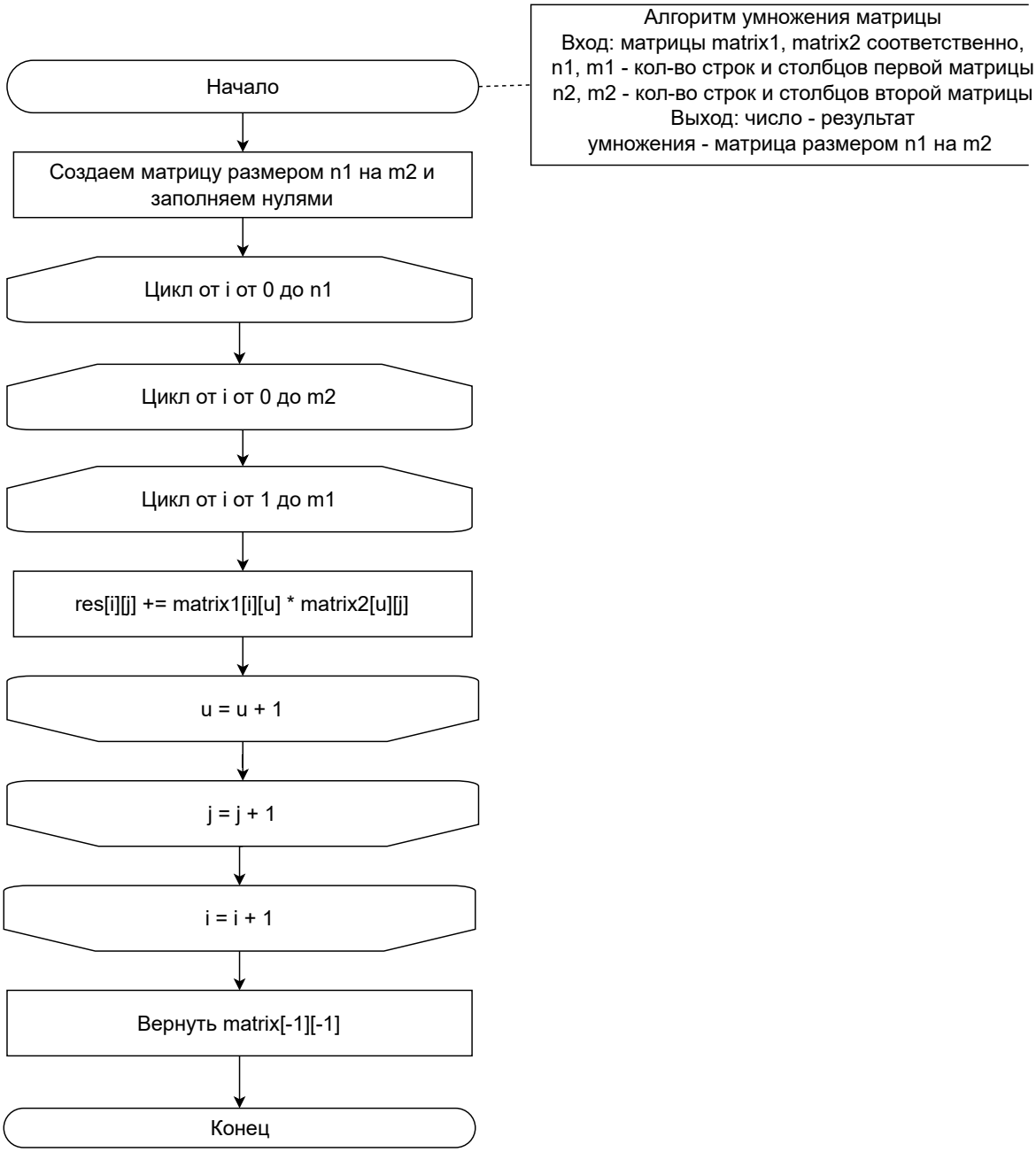


Рисунок 2.1 — Стандартный алгоритм умножения матриц.

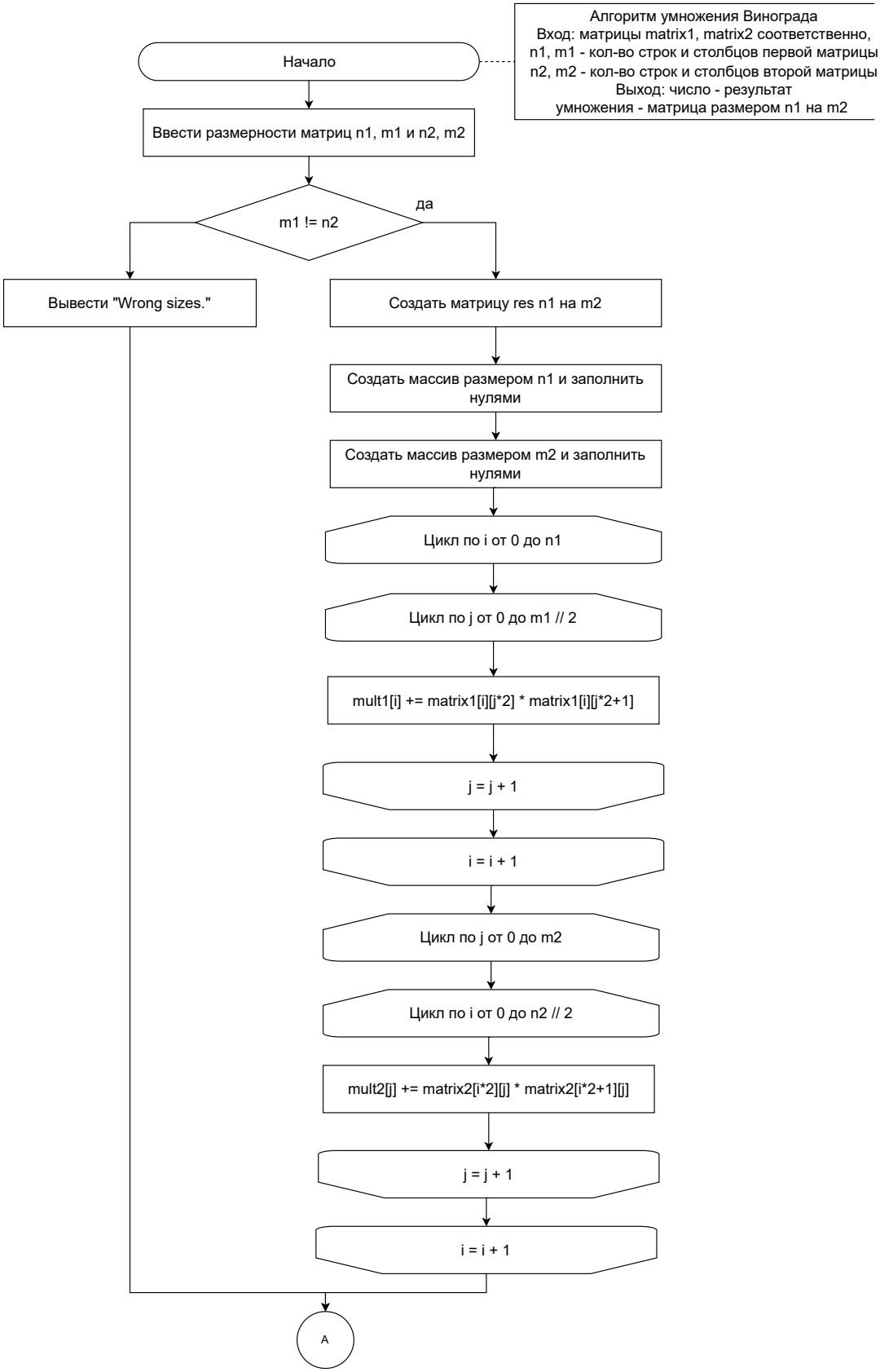


Рисунок 2.2 — Алгоритм Винограда, часть 1.

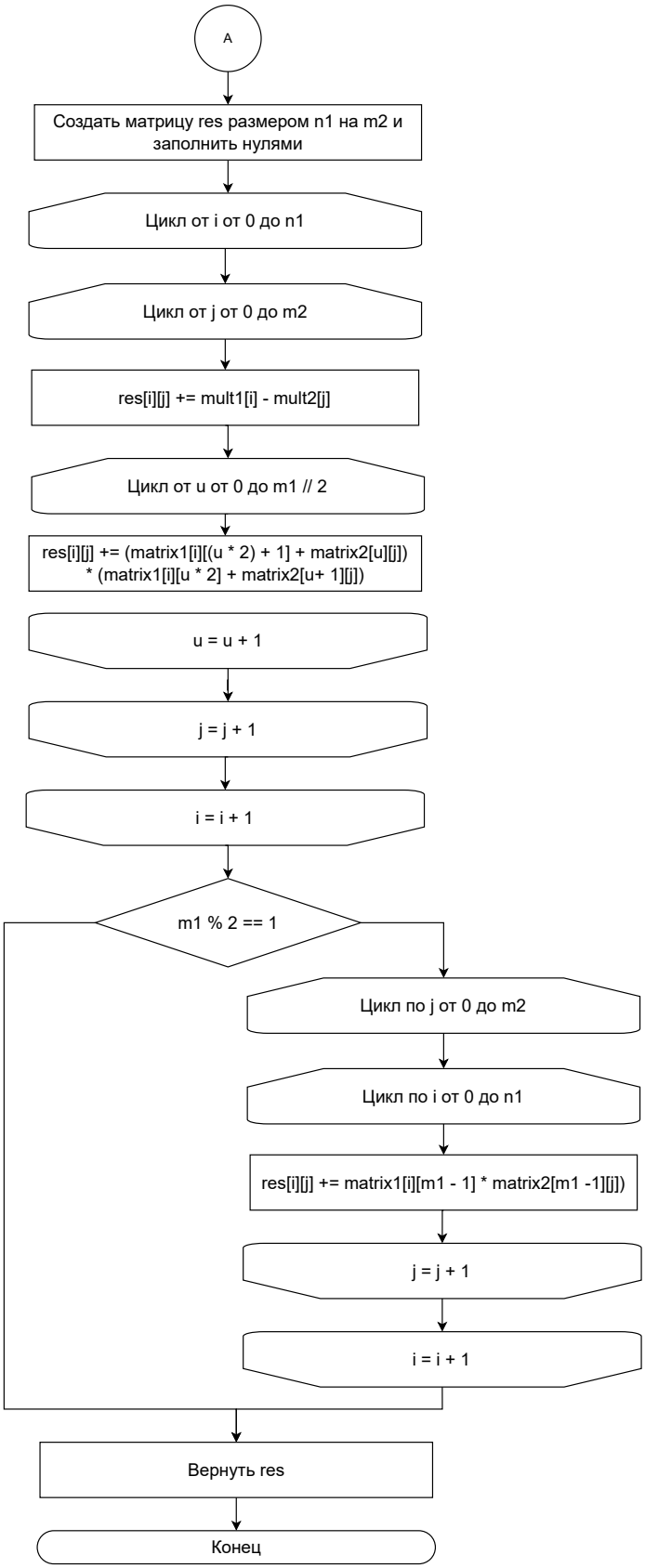


Рисунок 2.3 — Алгоритм Винограда, часть 2.

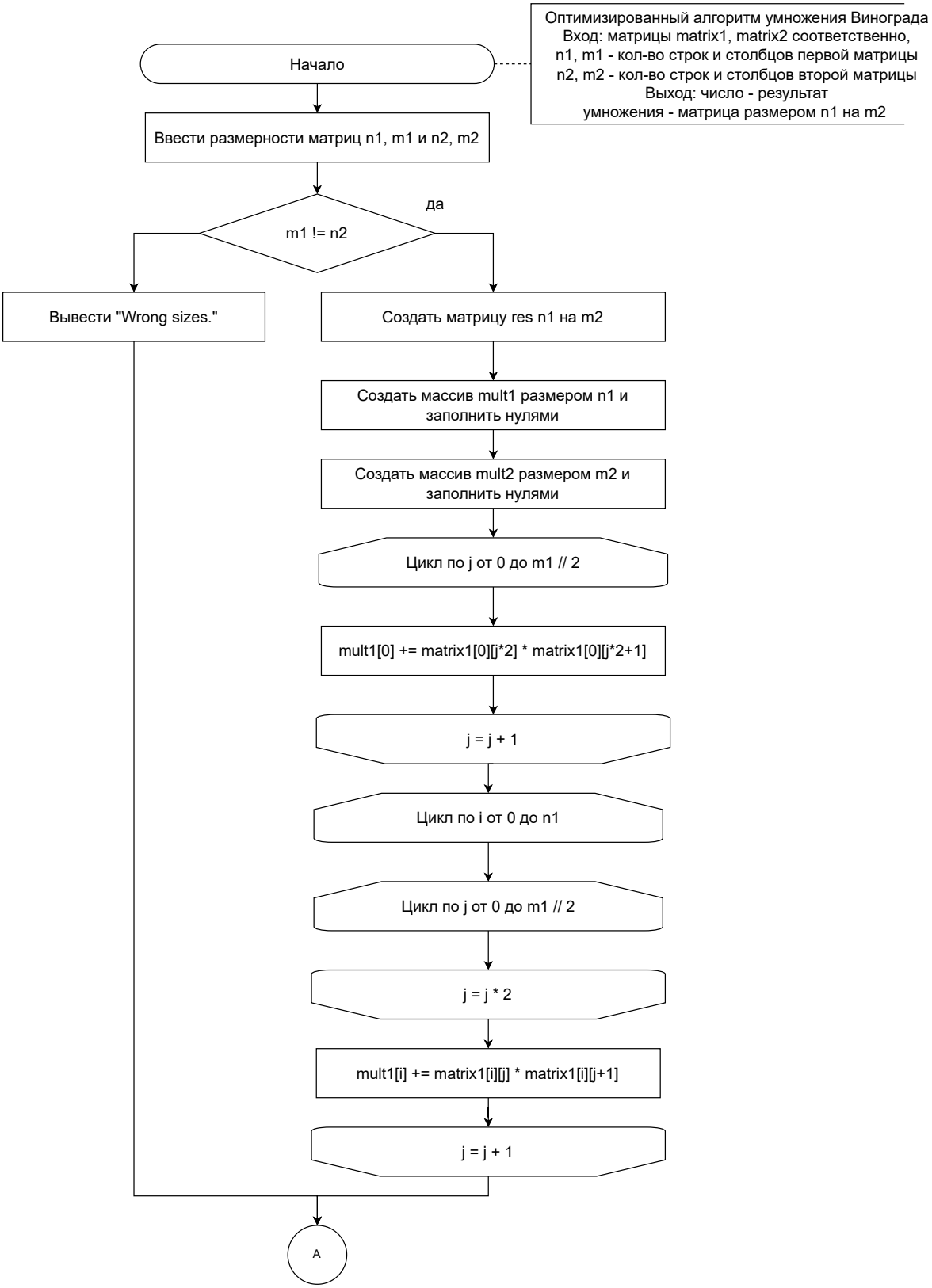


Рисунок 2.4 — Оптимизированный алгоритм Винограда, часть 1.

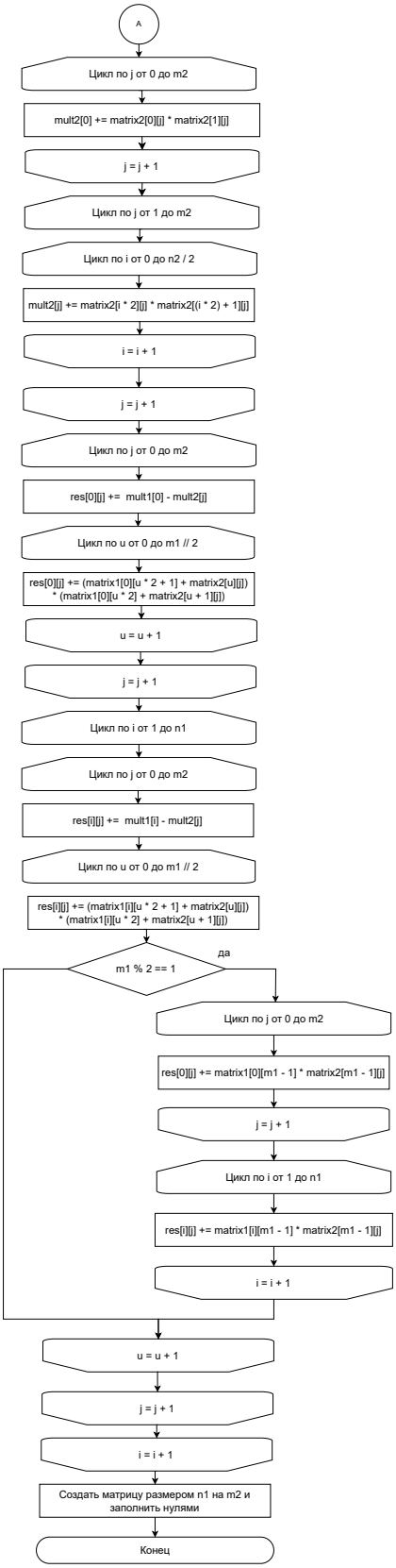


Рисунок 2.5 — Оптимизированный алгоритм Винограда, часть 2.

## 2.4 Модель вычисления трудоёмкости

Для оценки трудоёмкости алгоритмов предложена модель, в которой трудоёмкость операций определяется следующим образом:

– Операции типа `+`, `-=`, `+=`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `[]`, `++`, `-`, `&&`, `»`, `«`, `||`, `&`, `|` считаются с трудоёмкостью 1;

– Операции `*`, `/`, `%`, `*=`, `/=`, `%=` имеют трудоёмкость 2;

– Трудоёмкость выполнения условного оператора рассчитывается по следующей формуле (2.1):

$$f_{\text{if}} = f_{\text{условия}} + \begin{cases} \min(f_1, f_2), & \text{лучший случай,} \\ \max(f_1, f_2), & \text{иначе.} \end{cases} \tag{2.1}$$

– Трудоёмкость выполнения цикла определяется по формуле (2.2):

$$\begin{gathered} f_{\text{for}} = f_{\text{инициализация}} + f_{\text{сравнение}} + \\ + M_{\text{итераций}} \cdot (f_{\text{тело цикла}} + f_{\text{инкремент}} + f_{\text{сравнение}}) \end{gathered} \tag{2.2}$$

– Трудоёмкость передачи аргументов в функцию и возврат из функции принимаются равными нулю.

## 2.5 Трудоёмкость алгоритма умножения матриц

Необходимо рассмотреть трудоёмкость стандартного алгоритма умножения матриц.

### 2.5.1 Инициализация результирующей матрицы

Трудоёмкость инициализации матрицы, в которую записывается результат, вычисляется по следующей формуле:

$$f_{\text{иниц. матрицы}} = 2 + m_1 \cdot (2 + f_{n_2}) = 2 + m_1 \cdot (4 + n_2 \cdot 3) = 2 + 4 \cdot m_1 + 3 \cdot m_1 \cdot n_2, \tag{2.3}$$

где $m_1$ — количество строк, $n_2$ — количество столбцов результирующей матрицы.

### 2.5.2 Трудоёмкость стандартного алгоритма умножения матриц

Трудоёмкость самого умножения. Внешний цикл по $i$ даёт следующий вклад:

$$f_{\text{умнож.}} = 2 + m_1 \cdot (f_i + 2) = 2 + m_1 \cdot (2 + n_2 \cdot (2 + f_j) + 2) \tag{2.4}$$

С учётом всех вложенных циклов:

$$f_{\text{умнож.}} = 2 + m_1 \cdot (2 + n_2 \cdot (2 + 2 + n_1 \cdot (2 + f_k)) + 2) \tag{2.5}$$

Итоговая трудоёмкость умножения вычисляется следующим образом:

$$f_{\text{умнож.}} = 2 + 4 \cdot m_1 + 4 \cdot m_1 \cdot n_2 + 10 \cdot m_1 \cdot n_1 \cdot n_2 \approx 10 \cdot m_1 \cdot n_1 \cdot n_2 \tag{2.6}$$

### 2.5.3 Общая трудоёмкость

Общая трудоёмкость алгоритма, включая инициализацию, рассчитывается по формуле:

$$f_{\text{общ.}} = 10 \cdot m_1 \cdot n_1 \cdot n_2 + 2 \cdot m_1 \cdot m_2 + 3 \cdot m_1 \cdot n_2 + 6 \cdot m_1 + 4 \approx 10 \cdot m_1 \cdot n_1 \cdot n_2 \tag{2.7}$$

## 2.6 Трудоёмкость алгоритма Винограда умножения матриц

Трудоёмкость алгоритма Винограда для умножения матриц состоит из нескольких частей:

– создание массивов $mult1$ и $mult2$:

$$f_{mult1} = 3 \cdot n1 + 3 \cdot m1 + 4 \approx 3 \cdot n1 + 3 \cdot m1 \tag{2.8}$$

$$f_{mult2} = 3 \cdot n2 + 3 \cdot m2 + 4 \approx 3 \cdot n2 + 3 \cdot m2 \tag{2.9}$$

– заполнение массивов $mult1$ и $mult2$:

$$f_{mult1} = 2 + n1 \cdot (2 + \lfloor m1/2 \rfloor) \approx 6.5 \cdot n1 \cdot \lfloor m1/2 \rfloor \tag{2.10}$$

$$f_{mult2} = 2 + n2 \cdot (2 + \lfloor m2/2 \rfloor) \approx 6.5 \cdot n2 \cdot \lfloor m2/2 \rfloor \tag{2.11}$$

– инициализация матрицы–результата $res$:

$$f_{res} = m2 \cdot n1 \tag{2.12}$$

– заполнение матрицы для чётных индексов:

$$f_{even} = 2 + n1 \cdot (2 + m2 \cdot (6 + 2 + \lfloor m1/2 \rfloor)) \approx 11.5 \cdot n1 \cdot m2 \cdot \left\lfloor \frac{m1}{2} \right\rfloor \tag{2.13}$$

– дополнительный цикл для нечётных индексов:

$$f_{odd} = \begin{cases} 2, & \text{если } m1 \text{ чётное} \\ 4 + 4 \cdot n1 + 12 \cdot n1 \cdot m2, & \text{иначе} \end{cases} \tag{2.14}$$

Таким образом, трудоёмкость алгоритма Винограда в лучшем случае равна:

$$f_{best} \approx 11.5 \cdot n1 \cdot m2 \cdot \left\lfloor \frac{m1}{2} \right\rfloor \tag{2.15}$$

Трудоёмкость в худшем случае выражается следующим образом:

$$f_{\text{worst}} = 3 \cdot n_1 + 3 \cdot m_1 + 6.5 \cdot n_1 \cdot \left\lfloor \frac{m_1}{2} \right\rfloor + 6.5 \cdot n_2 \cdot \left\lfloor \frac{m_2}{2} \right\rfloor + m_2 \cdot n_1 + 11.5 \cdot n_1 \cdot m_2 + 4 + 4 \cdot n_1 + 12 \cdot n_1 \cdot m_2 \tag{2.16}$$

Или:

$$f_{worst} \approx 11.5 \cdot n1 \cdot m2 + 3 \cdot n1 + 3 \cdot m1 + 6.5 \cdot n2 \cdot \left\lfloor \frac{m2}{2} \right\rfloor + 4 \tag{2.17}$$

## 2.7 Трудоёмкость оптимизированного алгоритма Винограда умножения матриц

### 2.7.1 Оптимизации в алгоритме

В данном задании, согласно варианту, предусмотрены следующие дополнительные оптимизации:

– использование двоичного сдвига вместо умножения на 2, что позволяет выполнять вычисления быстрее на уровне процессора;

– объединение III и IV частей алгоритма Винограда для повышения эффективности по времени;

– вынос начальной итерации из внешних циклов, что сокращает количество операций внутри циклов и ускоряет выполнение программы.

## 2.8 Трудоёмкость оптимизированного алгоритма Винограда умножения матриц

### 2.8.1 Оптимизации в алгоритме

В данном задании, согласно варианту, предусмотрены следующие дополнительные оптимизации:

– использование двоичного сдвига вместо умножения на 2, что позволяет выполнять вычисления быстрее на уровне процессора;

– объединение III и IV частей алгоритма Винограда для повышения эффективности по времени;

– вынос начальной итерации из внешних циклов, что сокращает количество операций внутри циклов и ускоряет выполнение программы.

### 2.8.2 Трудоёмкость оптимизированного алгоритма Винограда

Трудоёмкость алгоритма Винограда для умножения матриц состоит из нескольких компонентов:

– создание массивов $mult1$ и $mult2$:

$$f = f_{mult1} + f_{mult2} = 3 \cdot m_1 + 3 \cdot n_2 + 4 \approx 3 \cdot m_1 + 3 \cdot n_2 \tag{2.18}$$

– заполнение $mult1$ и $mult2$:

$$f_{mult1} \approx 5 \cdot m_1 \cdot n_1 \tag{2.19}$$

$$f_{mult2} \approx 5 \cdot n_2 \cdot m_2 \tag{2.20}$$

– инициализация матрицы-результата:

$$f \approx n_2 \cdot m_1 \tag{2.21}$$

– цикл заполнения для чётных:

$$f = 9 \cdot n_1 \cdot n_2 \cdot m_1 \tag{2.22}$$

Таким образом, трудоёмкость в лучшем случае может быть выражена как:

$$f_{\text{лучший}} \approx 9 \cdot n_1 \cdot n_2 \cdot m_1 \tag{2.23}$$

В худшем случае трудоёмкость аналогична:

$$f_{\text{худший}} \approx 9 \cdot n_1 \cdot n_2 \cdot m_1 \tag{2.24}$$

Итоговая трудоёмкость:

$$f_{\text{итог}} \approx 9 \cdot n_1 \cdot n_2 \cdot m_1 \tag{2.25}$$

Таким образом, оптимизированный алгоритм Винограда имеет трудоёмкостью, приблизительно равной $9 \cdot n_1 \cdot n_2 \cdot m_1$.

## Вывод

В данном разделе разработаны описания алгоритмов умножения матриц, а также рассмотрены основные параметры для оценки их трудоёмкости. Описаны структуры данных, взаимодействие пользователя с программой, а также проведена теоретическая оценка трудоёмкости

классического алгоритма умножения матриц, алгоритма Винограда и его оптимизированной версии.

# 3 Технологическая часть

В данном разделе приведены данные о выбранном языке программирования, реализации алгоритмов и тесты для каждого алгоритма.

## 3.1 Требования к программному обеспечению

Для корректной работы программного обеспечения предусмотрены следующие требования:

– Входные данные: две матрицы $matrix1$ и $matrix2$. Количество столбцов в матрице $matrix1$ должно совпадать с количеством строк в матрице $matrix2$.

– Выходные данные: результат умножения входных матриц, представленный в матрице $result_matrix$.

## 3.2 Средства реализации

Для реализации алгоритмов, проведения замеров времени и построения графиков используется язык программирования `Python` [3]. `Python` предоставляет все необходимые инструменты для реализации матричных алгоритмов и построения графиков. В ходе разработки алгоритмов применяются следующие типы и структуры данных:

– двумерные матрицы типа `int`;

– переменные для хранения количества строк и столбцов в матрицах.

Замеры времени выполняются с помощью функции `process_time` [4].

## 3.3 Структура программы

Программа состоит из нескольких модулей:

– `main.py` – файл, который обеспечивает интерфейс и взаимодействие с пользователем, а также выполняет замеры времени работы алгоритмов;

– `algs.py` – файл, содержащий реализацию всех алгоритмов;

– `in_out.py` – модуль для ввода и вывода данных;

– `compare_time.py` – модуль для сравнения времени выполнения алгоритмов.

## 3.4 Реализация алгоритмов

В листинге 3.1 представлена реализация функции `insert_matrix`, которая создаёт матрицу, запрашивая ввод элементов с клавиатуры.

```
def insert_matrix(n: int, m: int):
    matrix = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
```

```
            matrix[i][j] = int(input())
    return matrix
```

Листинг 3.1 — Реализация создания матрицы через ввод с клавиатуры

В листинге 3.2 представлена реализация функции `create_random_matrix`, которая генерирует случайную матрицу с элементами от 0 до 100.

```
def create_random_matrix(n: int, m: int):
    matrix = [[random.randint(0, 100) for _ in range(m)] for _ in range
       (n)]
    return matrix
```

Листинг 3.2 — Реализация генерации случайной матрицы с элементами от 0 до 100

В листинге 3.3 представлена реализация функции `print_answer`, которая выводит матрицу на экран.

```
def print_answer(matrix, verbose=True):
    if not verbose:
        return
    for i, row in enumerate(matrix):
        print("  " + " ".join(f"{val:2}" for val in row))
```

Листинг 3.3 — Реализация вывода матрицы

В листинге 3.4 представлена реализация функции `multiply_base`, которая выполняет умножение матриц с использованием стандартного алгоритма.

```
def multiply_base(matrix1, matrix2):
    n1 = len(matrix1)
    m1 = len(matrix1[0])
    n2 = len(matrix2)
    m2 = len(matrix2[0])

    if m1 != n2:
        print("Wrong sizes.")
        return []

    res = [[0] * m2 for _ in range(n1)]

    for i in range(n1):
        for j in range(m2):
```

```
            for u in range(m1):
                res[i][j] += matrix1[i][u] * matrix2[u][j]
    return res
```

Листинг 3.4 — Реализация умножения матриц с использованием стандартного алгоритма

В листинге 3.5 представлена реализация функции `multiply_win`, которая выполняет умножение матриц с использованием алгоритма Винограда.

```
def multiply_win(matrix1, matrix2):
    n1 = len(matrix1)
    m1 = len(matrix1[0])
    n2 = len(matrix2)
    m2 = len(matrix2[0])

    if m1 != n2:
        print("Wrong sizes.")
        return []
    res = [[0] * m2 for _ in range(n1)]
    mult1 = [0] * n1
    for j in range(m1 // 2):
        mult1[0] += matrix1[0][j * 2] * matrix1[0][(j * 2) + 1]
    for i in range(1, n1, 1):
        for j in range(m1 // 2):
            mult1[i] += matrix1[i][j * 2] * matrix1[i][(j * 2) + 1]

    mult2 = [0] * m2
    for i in range(n2 // 2):
        mult2[0] += matrix2[i * 2][0] * matrix2[(i * 2) + 1][0]
    for j in range(1, m2, 1):
        for i in range(n2 // 2):
            mult2[j] += matrix2[i * 2][j] * matrix2[(i * 2) + 1][j]

    for j in range(m2):
        res[0][j] = mult1[0] - mult2[j]
        for u in range(m1 // 2):
            res[0][j] += (matrix1[0][(u * 2) + 1] + matrix2[u][j]) * \
                                (matrix1[0][u * 2] + matrix2[u + 1][j
                                   ])
    for i in range (1, n1, 1):
        for j in range(m2):
            res[i][j] = mult1[i] - mult2[j]
```

```
            for u in range(m1 // 2):
                res[i][j] += (matrix1[i][(u * 2) + 1] + matrix2[u][j])
                   * \
                                    (matrix1[i][u * 2] + matrix2[u + 1][j
                                       ])
    if m1 % 2 == 1:
        for j in range(m2):
            res[0][j] += matrix1[0][m1 - 1] * matrix2[m1 - 1][j]
        for i in range(1, n1, 1):
            for j in range(m2):
                res[i][j] += matrix1[i][m1 - 1] * matrix2[m1 - 1][j]
    return res
```

Листинг 3.5 — Реализация умножения матриц с использованием алгоритма Винограда

В листинге 3.6 представлена реализация функции `multiply_win_op`, которая выполняет умножение матриц, используя оптимизированный алгоритм Винограда.

```
def multiply_win_op(matrix1, matrix2):

    n1 = len(matrix1)
    m1 = len(matrix1[0])
    n2 = len(matrix2)
    m2 = len(matrix2[0])

    if m1 != n2:
        print("Wrong sizes.")
        return []

    res = [[0] * m2 for _ in range(n1)]

    mult1 = [0] * n1
    for j in range(m1 >> 1):
        mult1[0] += matrix1[0][j << 1] * matrix1[0][(j << 1) + 1]
    for i in range(1, n1):
        for j in range(m1 >> 1):
            mult1[i] += matrix1[i][j << 1] * matrix1[i][(j << 1) + 1]

    mult2 = [0] * m2
    for i in range(n2 >> 1):
        mult2[0] += matrix2[i << 1][0] * matrix2[(i << 1) + 1][0]
    for j in range(1, m2):
        for i in range(n2 >> 1):
```

```
            mult2[j] += matrix2[i << 1][j] * matrix2[(i << 1) + 1][j]

    for j in range(m2):
        res[0][j] = mult1[0] - mult2[j]
        for u in range(m1 >> 1):
            res[0][j] += (matrix1[0][(u << 1) + 1] + matrix2[u][j]) * \
                                (matrix1[0][u << 1] + matrix2[u + 1][
                                    j])
    for i in range(1, n1):
        for j in range(m2):
            res[i][j] = mult1[i] - mult2[j]
            for u in range(m1 >> 1):
                res[i][j] += (matrix1[i][(u << 1) + 1] + matrix2[u][j])
                    * \
                                (matrix1[i][u << 1] + matrix2[u + 1][
                                    j])
    if m1 % 2 == 1:
        for j in range(m2):
            res[0][j] += matrix1[0][m1 - 1] * matrix2[m1 - 1][j]
        for i in range(1, n1):
            for j in range(m2):
                res[i][j] += matrix1[i][m1 - 1] * matrix2[m1 - 1][j]
    return res
```

Листинг 3.6 — Реализация умножения матриц с использованием оптимизированного алгоритма Винограда

## 3.5 Функциональные тесты

В таблице 3.1 приведены результаты функционального тестирования для алгоритмов, реализующих умножение матриц. Все тесты пройдены успешно.

| **1-ая матрица** | **2-ая матрица** | **Ожидаемый результат** | **Фактический результат** |
|---|---|---|---|
| $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ | $\begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$ | $\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$ | $\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$ |
| $\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$ | $\begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$ | $\begin{pmatrix} 32 \end{pmatrix}$ | $\begin{pmatrix} 32 \end{pmatrix}$ |
| $\begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}$ | $\begin{pmatrix} 1 & 3 \\ 4 & 0 \end{pmatrix}$ | $\begin{pmatrix} 14 & 6 \\ 10 & 6 \end{pmatrix}$ | $\begin{pmatrix} 14 & 6 \\ 10 & 6 \end{pmatrix}$ |
| $\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$ | $\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ | $\begin{pmatrix} 0 \end{pmatrix}$ | $\begin{pmatrix} 0 \end{pmatrix}$ |
| $\begin{pmatrix} 0 & 0 \end{pmatrix}$ | $\begin{pmatrix} 0 & 0 \end{pmatrix}$ | $\begin{pmatrix} 0 \end{pmatrix}$ | $\begin{pmatrix} 0 \end{pmatrix}$ |
| $\begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}$ | $\begin{pmatrix} 2 & 3 \\ 1 & 0 \end{pmatrix}$ | $\begin{pmatrix} 9 & 18 \\ 10 & 14 \end{pmatrix}$ | $\begin{pmatrix} 9 & 18 \\ 10 & 14 \end{pmatrix}$ |

Таблица 3.1 — Функциональные тесты

## Вывод

В данном разделе представлены результаты тестирования алгоритмов умножения матриц. Все тесты прошли успешно, фактические результаты совпадают с ожидаемыми. Алгоритмы корректно обрабатывают ошибки при несовпадении размеров матриц.

# 4 Исследовательская часть

В данном разделе представлены результаты исследования эффективности работы алгоритмов по времени выполнения, основанной на критерии временных затрат.

## 4.1 Технические характеристики

Технические характеристики компьютера, на котором проводились замеры времени:

- операционная система: macOS Monterey;
- оперативная память: 8 ГБ;
- процессор: Quad-Core Intel Core i5 с тактовой частотой 1.4 ГГц;

## 4.2 Сравнительный анализ времени выполнения алгоритмов

Замеры проводились на матрицах размерностью от 20 до 100. Выбор размерностей обусловлен необходимостью анализа производительности алгоритмов на различных уровнях сложности, что выявляет их особенности и характеристики. Для достижения высокой точности полученных данных было выполнено по 100 замеров для каждой размерности. Результаты замеров представлены в таблицах 4.1 и 4.2. На рисунках 4.1 и 4.2 отображены графики, построенные на основе табличных данных.

| Размер | Классический | Виноград | Оптимизированный Виноград |
|---:|---:|---:|---:|
| 20 | 2.14e-03 | 2.46e-03 | 2.78e-03 |
| 30 | 7.12e-03 | 9.41e-03 | 9.04e-03 |
| 40 | 1.71e-02 | 1.83e-02 | 2.13e-02 |
| 50 | 3.33e-02 | 3.58e-02 | 4.15e-02 |
| 60 | 6.00e-02 | 6.00e-02 | 7.20e-02 |
| 70 | 9.24e-02 | 9.38e-02 | 1.12e-01 |
| 80 | 1.37e-01 | 1.42e-01 | 1.66e-01 |

Таблица 4.1 — Результаты измерений времени выполнения алгоритмов при чётной размерности матриц (в секундах).

| Размер | Классический | Виноград | Оптимизированный Виноград |
|---:|---:|---:|---:|
| 19 | 1.95e-03 | 2.14e-03 | 3.12e-03 |
| 29 | 7.24e-03 | 7.06e-03 | 8.26e-03 |
| 39 | 2.13e-02 | 1.90e-02 | 1.94e-02 |
| 49 | 3.04e-02 | 3.23e-02 | 3.76e-02 |
| 59 | 5.30e-02 | 5.60e-02 | 6.55e-02 |
| 79 | 1.26e-01 | 1.32e-01 | 1.60e-01 |
| 89 | 1.80e-01 | 1.92e-01 | 2.25e-01 |
| 99 | 2.47e-01 | 2.57e-01 | 3.02e-01 |

Таблица 4.2 — Результаты измерений времени выполнения алгоритмов при нечётной размерности матриц (в секундах).

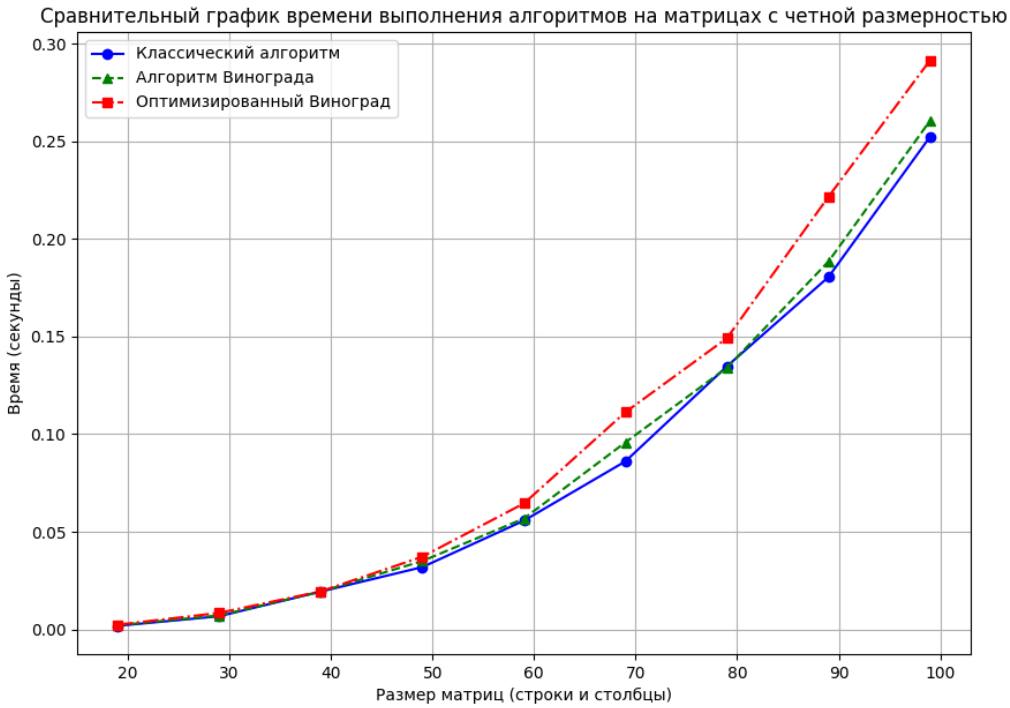


Рисунок 4.1 — Сравнительный график времени выполнения алгоритмов на матрицах с чётной размерностью.

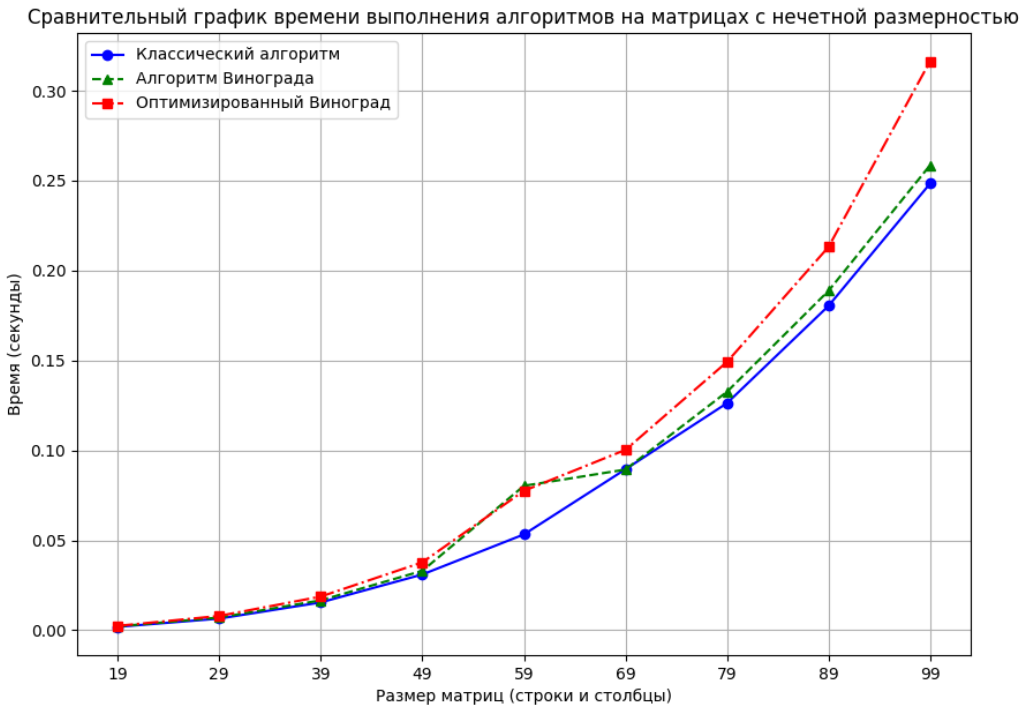


Рисунок 4.2 — Сравнительный график времени выполнения алгоритмов на матрицах с нечётной размерностью.

## Вывод

Стандартный алгоритм и алгоритм Винограда демонстрируют схожие временные затраты на матрицах размером до $300 \times 300$. Однако производительность алгоритма Винограда падает на больших размерностях (от $500 \times 500$). В отличие от них, оптимизированный алгоритм Винограда работает быстрее, и с увеличением размеров матриц разница в скорости становится более значимой.

Стандартный алгоритм Винограда показал себя менее эффективным, особенно на нечётных матрицах, где требуется больше дополнительных вычислений, в то время как оптимизированный алгоритм практически не теряет в скорости благодаря оптимизации.

Таким образом, оптимизированный алгоритм Винограда является самым быстрым решением для матриц размером (от $500 \times 500$). Стандартный алгоритм Винограда менее эффективен на нечётных матрицах из-за необходимости дополнительных вычислений. Классический алгоритм оказался самым медленным на матрицах от $500 \times 500$, но может быть полезен при решении небольших задач. Поэтому для оптимального умножения матриц предпочтителен оптимизированный алгоритм Винограда.

# ЗАКЛЮЧЕНИЕ

Цель работы достигнута. В ходе выполнения данной лабораторной работы решены следующие задачи:

– реализованы и проанализированы классический алгоритм умножения матриц, алгоритм Винограда и его оптимизированная версия;

– проведён анализ временных затрат алгоритмов умножения матриц на различных размерностях, что позволило выявить их особенности и различия в производительности;

– выполнен анализ трудоёмкости алгоритмов, учитывающий количество операций и требования к памяти;

– подготовлены графики, которые демонстрируют результаты сравнительного анализа;

– разработаны тесты для проверки правильности и эффективности работы реализованных алгоритмов по критериям временных затрат и использования памяти;

– представлены полученные результаты в виде отчёта о выполненной лабораторной работе.

Исследования показали, что оптимизированный алгоритм Винограда наиболее эффективен для умножения матриц размером от $500 \times 500$. Он быстрее алгоритма умножения матриц и стандартного алгоритма Винограда.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Голуб, Дж. & Ван Лоун, Ч. Матричные вычисления: Пер. с англ. — М.: Мир, 1999. — 548 с., ил. — ISBN 5-03-002406-9.

2. Coppersmith, D. & Winograd, S. Matrix multiplication via arithmetic progressions. В: Proceedings of the nineteenth annual ACM symposium on Theory of computing. — New York: ACM, 1987. — С. 1–6.

3. Python Software Foundation. Официальная документация по Python [Электронный ресурс]. — Режим доступа: `https://www.python.org/doc/` (дата обращения: 22.10.2024).

4. Python Software Foundation. Модуль time — Доступ к времени и его преобразования [Электронный ресурс]. — Режим доступа: `https://docs.python.org/3/library/time.html` (дата обращения: 22.10.2024).